

2 单元测试工具（JUnit+Mockito）

2.1 设计思路

2.1.1 什么是单元测试

单元测试作为软件开发的环节，主要针对程序中最基本的可测试单元进行验证。在 Java 项目中，这些单元通常对应单个方法或类。通过实施单元测试，开发者能够在编码早期识别潜在缺陷，确保各个独立模块的功能符合设计预期，从而为软件整体稳定性提供重要支撑。除了验证功能正确性之外，单元测试还在代码重构过程中扮演着安全网的角色，使开发者在优化代码结构时无需担忧对现有功能造成意外影响。

与集成测试、系统测试相比，单元测试具备多项显著特点。其执行效率较高，因测试对象不涉及外部依赖；问题定位精准，一旦测试失败可立即锁定具体方法；维护成本较低，需求变更时仅需调整少量相关用例；此外，单元测试还能充当活文档，通过测试用例清晰展示代码的预期行为。在敏捷开发与持续集成实践中，单元测试已成为关键环节，有效保障每次代码变更的质量可控性。

2.1.2 Junit 测试框架

Mockito 作为 Java 生态中首屈一指的模拟框架，其核心价值在于解决了单元测试的关键挑战：如何有效隔离被测试代码与其依赖项。在真实的业务代码中，一个类通常会依赖数据库操作、远程服务调用等复杂的外部组件。若在单元测试中直接使用这些真实依赖，将导致测试执行缓慢、结果不稳定。Mockito 通过创建模拟对象来替代这些真实依赖，使测试能够在纯净、可控的环境下运行，从而保证了测试的快速性、可靠性和可重复性。在本项目中，诸如 UserRepository、FileRepository 等数据访问层组件以及其他服务层依赖，都通过 Mockito 进行了模拟。

从技术原理上看，Mockito 利用动态代理或字节码增强技术，在运行时为指定接口或类生成代理对象。当使用 @Mock 注解标记一个字段时，Mockito 会创建这样一个代理实例。其核心功能之一是“打桩”，即通过 when().thenReturn() 这样的语法来预设模拟对象的行为。例如，when(userRepository.findById(1L)).thenReturn(optionalUser) 就明确指定了当使用参数 1L

调用 `findById` 方法时应返回的结果。另一项关键功能是“验证”，通过 `verify()` 方法可以断言模拟对象的某个方法是否被调用、调用次数以及参数是否正确，这对于检验类之间的交互逻辑至关重要。

为了高效运用 Mockito，遵循一些最佳实践是十分必要的。首先，应合理使用 `@Mock` 和 `@InjectMocks` 注解。`@Mock` 用于声明待模拟的依赖，而 `@InjectMocks` 则会自动将模拟的依赖注入到被测试类的实例中，这大大简化了测试的设置。其次，应避免过度模拟，对于不包含复杂逻辑或外部交互的简单对象（如值对象），直接使用真实实例可能更合适。再次，进行打桩时应充分考虑各种场景，包括正常路径、边界情况和异常路径，以确保测试的完备性。最后，需要注意 Mockito 的局限性，例如它默认无法模拟 `final` 类、`final` 方法及静态方法，在面对这些情况时可能需要调整代码设计或借助其他工具。在本项目的测试实践中，我们严格遵循了这些原则，以保障测试代码的质量与可维护性。

2.1.3 Mockito 框架原理

Mockito 是 Java 领域内占据主导地位的测试模拟框架，其根本使命是破解单元测试的核心挑战：如何将被测代码与其依赖项有效隔离开来。现实中的业务类常常耦合了数据库、网络服务或文件系统等外部资源，若在测试中直接调用这些真实依赖，会导致测试用例执行缓慢、结果不可靠。Mockito 通过生成模拟对象来充当这些依赖的替身，让测试能够在独立、可控的沙箱环境中运行，从而达成高效、稳定、可重复的验证目标。在本项目里，我们普遍采用 Mockito 来模拟数据访问层组件（如 `UserRepository`、`FileRepository`）以及其他服务层依赖对象。

从实现机制上说，Mockito 主要借助动态代理或字节码操纵技术。当某个字段被标记为 `@Mock` 时，框架会在运行时动态创建一个代理对象，该对象与原始类接口一致，但其行为完全由测试者定义。其中，“打桩”是 Mockito 的核心能力，通过形如 `when(...).thenReturn(...)` 的链式调用，开发者可预定义模拟对象在特定输入下的返回值。举例来说，`when(userRepository.findByUsername("admin")).thenReturn(user)` 就指明了当查询用户名为 "admin" 时应返回预设的用户对象。另一项关键功能是“行为验证”，借助 `verify()` 方法可断言某个模拟方法是否被调用、调用次数及参数内容，这对检验对象间的协作逻辑至关重要。

要有效运用 Mockito，需遵循若干最佳实践。首要的是善用 `@Mock` 与 `@InjectMocks` 注

解：前者用于标记待模拟的依赖项，后者则自动将这些模拟实例注入到被测试对象中，极大简化了测试初始化流程。其次应警惕“过度模拟”，并非所有依赖都需模拟——例如不含外部交互逻辑的值对象或工具类，直接实例化更为简洁。另外，打桩时应充分考虑正常流程、边界情况及异常场景，以保证测试覆盖的全面性。还需留意 Mockito 的固有局限，比如默认无法模拟 final 类/方法、静态方法等，此时可能需要调整代码结构或引入辅助工具。在本项目测试实践中，我们严格遵循上述准则，从而保障了测试代码的健壮性与可维护性。

2.2 实现过程

2.2.1 环境配置与依赖管理

在着手编写单元测试之前，进行正确的环境配置是首要步骤。本项目基于 Maven 构建工具和 Spring Boot 应用框架，因此核心配置在于 pom.xml 文件。我们选用当前功能最成熟、完善的 JUnit 5 (JUnit Jupiter) 和 Mockito 3.x 作为测试框架。为了简化依赖管理，只需引入 spring-boot-starter-test 即可，因为它是一个聚合依赖，自动囊括了 JUnit、Mockito、AssertJ 等一整套测试所需的库，避免了逐个引入的繁琐。同时，必须正确配置 JaCoCo 插件，以确保在构建过程中能够生成清晰的测试覆盖率报告，便于量化评估测试效果。在开发工具层面，IntelliJ IDEA 为这些测试框架提供了开箱即用的支持，开发者不仅能直接运行测试、直观查看结果，还能利用其强大的代码生成功能，通过快捷键快速创建测试类和测试方法的基本结构，从而提升效率。

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals>
        <goal>report</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

图 2.2.1 环境配置与依赖管理

2.2.2 基础单元测试实现-日期验证工具类

本项目的 DateValidator 日期验证工具类，用于演示白盒测试覆盖准则而构建的示例。具

体而言，`validateDate` 方法的执行流程如下：它首先校验月份是否在 1 到 12 之间，接着检查日期是否在 1 到 31 之间。之后，逻辑会进入更细致的判断：对于 2 月，需要根据复杂的闰年规则（同时满足被 4 整除、且不被 100 整除，或者直接被 400 整除）来确定其最大天数；而对于 4 月、6 月、9 月、11 月这几个小月，则直接限定为 30 天。这种由多个 if 条件分支构成的、环环相扣的逻辑结构，天然地创造了丰富的测试路径，非常适合用来设计和讲解如何实现语句覆盖、分支覆盖、条件组合覆盖等白盒测试目标。

```
package com.ai.project.util;
/**
 * 日期验证工具类
 * 用于验证日期是否符合白盒测试
 * @author 11591
 */
public class DateValidator {
    /**
     * 验证日期是否有效
     * @param year 年份
     * @param month 月份
     * @param day 日期
     * @return 验证结果，"有效日期" 或 "无效日期:xxx"
     */
    public static String validateDate(int year, int month, int day) {
        // 判断月份是否有效
        if (month < 1 || month > 12) {
            return "无效日期:月份超出范围";
        }
        // 判断日期是否有效
        if (day < 1 || day > 31) {
            return "无效日期:日期超出范围";
        }
        // 判断闰年
        if (month == 2) {
            boolean isLeapYear = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
            if (isLeapYear && day > 29) {
                return "无效日期:闰年2月最多29天";
            } else if (!isLeapYear && day > 28) {
                return "无效日期:平年2月最多28天";
            }
        } else if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30) {
            return "无效日期:该月份最多30天";
        }
        return "有效日期";
    }
}
```

图 2.2.2 DataValidator 类代码

2.2.3 白盒测试与覆盖准则实践

白盒测试是一种基于程序内部代码结构和逻辑路径的测试方法，要求测试者知晓代码的具体实现，并据此设计用例来验证不同的程序执行流程。在对 `DateValidator` 进行测试时，我们重点运用了三种覆盖标准：语句覆盖、判定覆盖和基本路径覆盖。其中，控制流图是分析程序结构的重要辅助工具，它用图形化方式呈现代码中的判定节点（菱形）和执行边，帮助我们系统性地识别所有需要覆盖的路径与分支，从而构建出完备的测试用例集合。

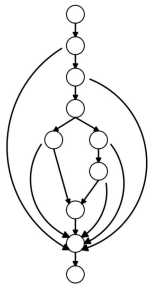


图 2.2.3(a) DataValidator 控制流图

```
@Test
@DisplayName("语句覆盖测试 - 月份超出范围 (小于1)")
void testValidateDate_MonthOutOfRange_LessThan1() {
    String result = DateValidator.validateDate(2024, 0, 15);
    assertEquals("预期: 无效日期:月份超出范围", result, "message: 月份小于1应为无效日期");
}

@Test
@DisplayName("语句覆盖测试 - 闰年2月29天 (有效)")
void testValidateDate_LeapYear_February29_Valid() {
    String result = DateValidator.validateDate(2024, 2, 29);
    assertEquals("预期: 有效日期", result, "message: 闰年2月29天应为有效日期");
}

@Test
@DisplayName("语句覆盖测试 - 平年2月29天 (无效)")
void testValidateDate_NonLeapYear_February29_Invalid() {
    String result = DateValidator.validateDate(2023, 2, 29);
    assertEquals("预期: 无效日期:平年2月最多28天", result, "message: 平年2月29天应为无效日期");
}
```

图 2.2.3(b) 语句覆盖测试代码

语句覆盖是其中最基础的准则，目标是确保程序中的每一条可执行语句至少被执行一次。在测试 `DateValidator` 时，我们设计了多组用例，分别覆盖诸如月份小于 1 或大于 12、日期小于 1 或大于 31、闰年 2 月 29 日与 30 日、平年 2 月 28 日与 29 日、30 天月份的有效与无效输入、以及 31 天月份等情况。借助这些用例，我们实现了对 `validateDate` 方法中每一条返回语句的执行，达到了 100% 的语句覆盖率。

判定覆盖（也称分支覆盖）比语句覆盖更强，它要求每个逻辑判断的“真”和“假”两个分支都至少执行一次。在 `DateValidator` 中，关键判定包括月份是否越界、日期是否越界、是否为 2 月、是否为闰年（规则：能被 4 整除但不能被 100 整除，或能被 400 整除）、以及是否为 30 天月份等。为满足判定覆盖，我们设计了相应用例，使每个判定的两种结果都被触发。例如在测试闰年条件时，我们选取了 2024 年（满足 4 整除但不被 100 整除）、2000 年（满足 400 整除）、1900 年（被 100 整除但不被 400 整除）以及 2023 年（不满足 4 整除），从而全面覆盖了闰年判定的所有分支。

```
@Test
@DisplayName("判定覆盖测试 - 月份边界值")
void testValidateDate_MonthBoundaryValues() {
    // 边界值: 月份=1 (有效)
    assertEquals("有效日期", DateValidator.validateDate(year: 2024, month: 1, day: 15));
    // 边界值: 月份=12 (有效)
    assertEquals("有效日期", DateValidator.validateDate(year: 2024, month: 12, day: 15));
    // 边界值: 月份=0 (无效)
    assertEquals("无效日期: 月份超出范围", DateValidator.validateDate(year: 2024, month: 0, day: 15));
    // 边界值: 月份=13 (无效)
    assertEquals("无效日期: 月份超出范围", DateValidator.validateDate(year: 2024, month: 13, day: 15));
}

@Test
@DisplayName("判定覆盖测试 - 闰年判定逻辑")
void testValidateDate_LeapYearLogic() {
    // 能被4整除但不能被100整除 -> 闰年
    assertEquals("有效日期", DateValidator.validateDate(year: 2024, month: 2, day: 29));
    // 能被400整除 -> 闰年
    assertEquals("有效日期", DateValidator.validateDate(year: 2000, month: 2, day: 29));
    // 能被100整除但不能被400整除 -> 平年
    assertEquals("无效日期: 平年2月最多28天", DateValidator.validateDate(year: 1900, month: 2, day: 29));
    // 不能被4整除 -> 平年
    assertEquals("无效日期: 平年2月最多28天", DateValidator.validateDate(year: 2023, month: 2, day: 29));
}
```

图 2.2.3(c) 判定覆盖测试代码

```
@Test
@DisplayName("基本路径覆盖测试 - 覆盖所有独立路径")
void testValidateDate_AllPaths() {
    // 路径1: 月份无效 (< 1)
    assertEquals("无效日期: 月份超出范围", DateValidator.validateDate(year: 2024, month: 0, day: 15));
    // 路径2: 月份无效 (> 12)
    assertEquals("无效日期: 月份超出范围", DateValidator.validateDate(year: 2024, month: 13, day: 15));
    // 路径3: 日期无效 (< 1)
    assertEquals("无效日期: 日期超出范围", DateValidator.validateDate(year: 2024, month: 6, day: 0));
    // 路径4: 日期无效 (> 31)
    assertEquals("无效日期: 日期超出范围", DateValidator.validateDate(year: 2024, month: 6, day: 32));
    // 路径5: 2月特殊处理
    assertEquals("有效日期", DateValidator.validateDate(year: 2024, month: 2, day: 29));
    // 路径6: 2月闰年无效 (> 29)
    assertEquals("无效日期: 闰年2月最多29天", DateValidator.validateDate(year: 2024, month: 2, day: 30));
    // 路径7: 2月平年有效
    assertEquals("有效日期", DateValidator.validateDate(year: 2023, month: 2, day: 28));
    // 路径8: 2月平年无效 (> 28)
    assertEquals("无效日期: 平年2月最多28天", DateValidator.validateDate(year: 2023, month: 2, day: 29));
    // 路径9: 30天月份有效
    assertEquals("有效日期", DateValidator.validateDate(year: 2024, month: 4, day: 30));
    // 路径10: 30天月份无效 (> 30)
    assertEquals("无效日期: 该月份最多30天", DateValidator.validateDate(year: 2024, month: 4, day: 31));
    // 路径11: 31天月份有效
    assertEquals("有效日期", DateValidator.validateDate(year: 2024, month: 1, day: 31));
}
```

图 2.2.3(d) 基本路径覆盖测试代码

基本路径覆盖是更为彻底的覆盖准则，它要求测试程序中的所有独立路径，即每条路径至少包含一条未被其他路径覆盖的边。通过分析控制流图，我们在 `DateValidator` 中识别出 11 条独立路径，包括：月份小于 1、月份大于 12、日期小于 1、日期大于 31、闰年 2 月有效、闰年 2 月无效（超过 29 天）、平年 2 月有效、平年 2 月无效（超过 28 天）、30 天月份有效、30 天月份无效（超过 30 天）、以及 31 天月份有效。通过设计用例覆盖这 11 条路径，我们不仅实现了语句和判定层面的全覆盖，还验证了所有可能的程序执行序列，显著增强了测试的充分性与代码的可信度。

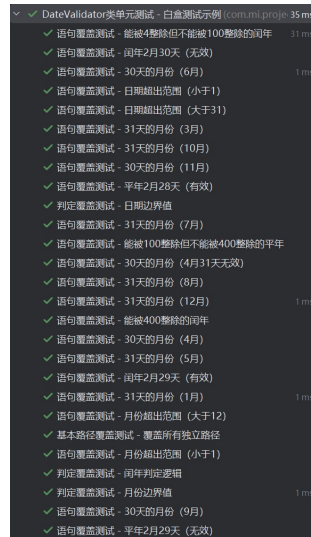


图 2.2.3(e) 基本路径覆盖测试代码

2.2.4 使用 Mockito 进行依赖打桩– UserService 测试

当被测试的业务类（如 UserService）依赖于其他组件（如数据访问层的 UserRepository）时，就需要借助 Mockito 来模拟这些依赖关系。在本项目的电力线分析系统中，UserService 包含了用户管理的核心逻辑，但其功能依赖于 UserRepository 来完成数据库操作。为了构建快速、稳定且不依赖外部数据库的单元测试，我们使用 Mockito 创建 UserRepository 的模拟对象，并预设其方法在各种场景下的行为，从而将测试焦点隔离在 UserService 本身的逻辑上。在测试类 UserServiceTest 中，我们通过@ExtendWith(MockitoExtension.class)注解启用 Mockito 框架支持。使用@Mock 注解来标记需要被模拟的 UserRepository 依赖，而@InjectMocks 注解则会让 Mockito 自动将这些模拟对象注入到被测试的 UserServiceImpl 实例中，极大简化了测试环境的搭建。

```
package com.wj.project.service.serviceimpl;
import org.junit.jupiter.api.*;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;

/**
 * UserService 单元测试
 * 使用Mockito进行依赖打桩
 */
@ExtendWith(MockitoExtension.class)
@DisplayName("UserService单元测试")
class UserServiceTest {
    @Mock // 65个用法
    private UserRepository userRepository;
    @InjectMocks // 15个用法
    private UserServiceImpl userService;
    private User testUser; // 5个用法
    private UserRegisterDTO registerDTO; // 5个用法

    @BeforeEach // 1个用法
    void setUp() {
        testUser = User.builder()
            .id(1L)
            .userName("testuser")
            .email("test@example.com")
            .phoneNumber("13888888888")
            .isActive(true)
            .createTime(LocalDateTime.now())
            .build();
        registerDTO = UserRegisterDTO.builder()
            .userName("testuser")
            .password("Password123!")
            .email("test@example.com")
            .phoneNumber("13888888888")
            .build();
    }
}
```

图 2.2.4(a) UserServiceTest 测试类结构

以用户注册功能为例，测试需要模拟 UserRepository 的多个交互：首先检查用户名是否

存在的 `existsByUsername`，然后是检查邮箱的 `existsByEmail` 和检查手机号的 `existsByPhoneNumber`，最后是保存用户的 `save` 方法。通过 `when().thenReturn()` 链，我们可以精确设定这些模拟方法的返回值。例如，`when(userRepository.existsByUsername("testuser")).thenReturn(false)` 表示当查询用户名 "testuser" 时返回“不存在”。当所有前置检查通过后，可以预设 `save` 方法返回一个预构造的测试用户对象。这种打桩方式使测试完全可控，执行迅速且结果可预测。

```
@Test
@DisplayName("用户注册 - 成功场景")
void testRegister_Success() {
    // Mock行为: 用户名、邮箱、手机号都不存在
    when(userRepository.existsByUsername("testuser")).thenReturn(value: false);
    when(userRepository.existsByEmail("test@example.com")).thenReturn(value: false);
    when(userRepository.existsByPhoneNumber("13888888888")).thenReturn(value: false);
    when(userRepository.save(any(User.class))).thenReturn(testUser);
    // 执行测试
    User result = userService.register(registerDTO);
    // 验证结果
    assertNotNull(result);
    assertEquals(expected: "testuser", result.getUserName());
    assertEquals(expected: "test@example.com", result.getEmail());
    // 验证方法调用
    verify(userRepository, times(wantedNumberOfInvocations: 1)).existsByUsername("testuser");
    verify(userRepository, times(wantedNumberOfInvocations: 1)).existsByEmail("test@example.com");
    verify(userRepository, times(wantedNumberOfInvocations: 1)).existsByPhoneNumber("13888888888");
    verify(userRepository, times(wantedNumberOfInvocations: 1)).save(any(User.class));
}
```

图 2.2.4(b) UserService 注册成功测试

一个完整的测试策略不仅要覆盖正常流程，还需验证各种异常情况。对于注册功能，异常场景包括用户名、邮箱或手机号已存在等。针对这些情况，我们使用 `thenThrow()` 方法来模拟依赖方法抛出特定业务异常，例如 `when(userRepository.existsByUsername("existingUser")).thenThrow(new UsernameExistsException())`。随后，使用 `assertThrows()` 来断言服务方法是否按预期抛出了对应的异常。此外，利用 `verify()` 方法可以确认在异常路径下，某些不应被调用的方法（如 `save`）确实未被执行，从而确保程序流程的正确性。这种全面的测试方案显著提升了服务类在处理各种边界和异常情况时的健壮性。

```
@Test
@DisplayName("用户注册 - 用户名已存在")
void testRegister_UserNameExists() {
    // Mock行为: 用户名已存在
    when(userRepository.existsByUsername("testuser")).thenReturn(value: true);
    // 执行测试并验证异常
    assertThrows(UserException.UserNameExistsException.class, () -> {
        userService.register(registerDTO);
    });
    // 验证方法调用
    verify(userRepository, times(wantedNumberOfInvocations: 1)).existsByUsername("testuser");
    verify(userRepository, never()).existsByEmail(anyString());
    verify(userRepository, never()).save(any(User.class));
}

@Test
@DisplayName("用户注册 - 邮箱已存在")
void testRegister_EmailExists() {
    // Mock行为: 用户名不存在，但邮箱已存在
    when(userRepository.existsByUsername("testuser")).thenReturn(value: false);
    when(userRepository.existsByEmail("test@example.com")).thenReturn(value: true);
    // 执行测试并验证异常
    assertThrows(UserException.EmailExistsException.class, () -> {
        userService.register(registerDTO);
    });
    // 验证方法调用
    verify(userRepository, times(wantedNumberOfInvocations: 1)).existsByUsername("testuser");
    verify(userRepository, times(wantedNumberOfInvocations: 1)).existsByEmail("test@example.com");
    verify(userRepository, never()).save(any(User.class));
}
```

图 2.2.4(c) UserService 异常场景测试

2.2.5 Controller 层测试-使用 MockMvc

Controller 层作为 Web 应用程序的入口点，承担着接收 HTTP 请求并返回相应响应的职责。为了有效测试这一层，可以利用 Spring 测试框架中的 MockMvc 工具，它能够在无需启动完整 Web 容器的情况下，模拟 HTTP 请求并验证响应行为，从而实现快速且可靠的 Controller 层测试。在 UserControllerTest 测试类中，我们通过 setUp 方法进行初始化，使用 MockMvcBuilders.standaloneSetup() 构建一个独立的测试环境，仅加载被测试的 Controller，避免加载整个 Spring 应用上下文，从而显著提升测试执行效率。同时，我们还配置了 ObjectMapper 用于处理 JSON 数据的转换，并设置了相应的消息转换器。

```
package com.mi.project.controller;

import org.junit.jupiter.api.*;
import org.mockito.*;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;
import org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@ExtendWith(MockitoExtension.class)
@DisplayName("UserController 单元测试")
class UserControllerTest {

    @Mock // 1 个用法
    private UserServiceImpl userService;
    @InjectMocks // 1 个用法
    private UserController userController;
    private MockMvc mockMvc; // 7 个用法
    private ObjectMapper objectMapper; // 9 个用法

    @BeforeEach // Huaflower2004
    void setUp() {
        objectMapper = new ObjectMapper();
        objectMapper.registerModule(new JavaTimeModule());
        objectMapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
        // 忽略未知字段，避免反序列化时出错
        objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);

        mockMvc = MockMvcBuilders.standaloneSetup(userController)
            .setMessageConverters(new MappingJackson2HttpMessageConverter(objectMapper))
            .build();
    }
}
```

图 2.2.5(a) UserControllerTest 测试类结构

```
@PostMapping("/register") // Huaflower2004
@Operation(summary = "用户注册", description = "注册新用户账号")
@RequestBody
public Result<User> register(@Valid @RequestBody UserRegisterDTO userRegisterDTO) {
    try {
        if (!userRegisterDTO.isPasswordMatching()) {
            throw new UserException.PasswordNotMatchException();
        }
        User user = userService.register(userRegisterDTO);
        return Result.success(user);
    } catch (Exception e) {
        return Result.failure(code: 500, message: "服务器错误" + e.getMessage());
    }
}
```

图 2.2.5(b) UserController 注册接口代码

使用 MockMvc 进行 Controller 测试通常包含四个关键步骤：构建请求、执行请求、验证响应结果以及验证方法调用。以用户注册接口测试为例，首先准备测试数据 UserRegisterDTO 对象，然后通过 mockMvc.perform() 方法构建一个 POST 请求，指定请求路径为 "/api/user/register"，设置 Content-Type 为 application/json，并将测试数据转换为 JSON 字符串作为请求体。执行请求后，通过 andExpect() 方法链验证响应内容，包括 HTTP 状态码是否为 200，以及响应体中如 .data.userName 是否为 "testuser" 等 JSON 字段值是否符合预期。最后，通过 verify() 方法确认 UserService 的 register 方法被正确调用一次，且传入参数类型正确。这种方式既检验了 Controller 的请求处理逻辑，也确保了其与 Service 层的正确协作。

```

@Test
@DisplayName("用户注册 - 成功场景")
void testRegister_Success() throws Exception {
    // 准备测试数据
    UserRegisterDTO registerDTO = UserRegisterDTO.builder()
        .userName("testUser")
        .password("Password123!")
        .confirmPassword("Password123!")
        .email("test@example.com")
        .phoneNumber("13888888888")
        .build();

    User savedUser = User.builder()
        .id(1L)
        .userName("testUser")
        .email("test@example.com")
        .phoneNumber("13888888888")
        .isActive(true)
        .createTime(LocalDate.now())
        .build();

    // Mock 行为
    when(userService.register(any(UserRegisterDTO.class))).thenReturn(savedUser);
    // 执行测试
    mockMvc.perform(post(URI, "/api/user/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(registerDTO)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.code").value(expectedValue: 200))
        .andExpect(jsonPath("$.data.userName").value(expectedValue: "testUser"))
        .andExpect(jsonPath("$.data.email").value(expectedValue: "test@example.com"));

    // 验证方法调用
    verify(userService, times(wantedNumberOfInvocations: 1)).register(any(UserRegisterDTO.class));
}

```

图 2.2.5(c) Controller 注册成功测试

Controller 层测试还需全面覆盖各类异常场景和边界条件。例如，测试当用户提交的密码与确认密码不一致时，Controller 应返回错误响应且不调用 Service 层；测试当 Service 层抛出 `UserNameExistsException` 异常时，Controller 应正确捕获并返回用户友好的错误信息；测试当请求参数不符合 `@Valid` 注解定义的校验规则（如用户名过短、邮箱格式错误、密码强度不足等）时，Controller 应返回详细的校验错误信息。通过实施这些全面的测试用例，我们能够确保 Controller 层在各种情况下均能正确响应，为用户提供稳定、友好的交互体验。

```

@Test
@DisplayName("用户注册 - 密码不匹配")
void testRegister_PasswordNotMatch() throws Exception {
    // 准备测试数据
    UserRegisterDTO registerDTO = UserRegisterDTO.builder()
        .userName("testUser")
        .password("Password123!")
        .confirmPassword("DifferentPassword123!")
        .email("test@example.com")
        .build();

    // 执行测试
    // 注意：密码不匹配时，Controller 会在调用 service 之前抛出异常
    mockMvc.perform(post(URI, "/api/user/register")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(registerDTO)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.code").value(expectedValue: 500))
        .andExpect(jsonPath("$.message").value(org.hamcrest.Matchers.containsString("服务器错误")));

    // 验证方法调用 - 密码不匹配时不会调用 userService.register
    verify(userService, never()).register(any(UserRegisterDTO.class));
}

```

✓ UserController单元测试 (com.mi.project.controller)	3 sec 377 ms
✓ 检查用户名 - 可用	3 sec 50 ms
✓ 用户注册 - 密码不匹配	197 ms
✓ 用户注册 - 用户名已存在	36 ms
✓ 检查用户名 - 不可用	21 ms
✓ 用户注册 - 邮箱已存在	26 ms
✓ 用户注册 - 成功场景	47 ms

图 2.2.5(e) Controller 测试执行结果

图 2.2.5(d) Controller 密码不匹配测试

2.2.6 JaCoCo 测试覆盖统计

JaCoCo（Java Code Coverage）是 Java 生态中广泛使用的代码覆盖率检测工具，它能够在测试执行过程中动态收集代码执行数据，并生成可视化的覆盖率报告。在 IntelliJ IDEA 中集成 JaCoCo 十分便捷：首先进入“设置”中的“构建、执行、部署”→“覆盖率”选项，将测试覆盖率运行器切换为 JaCoCo。配置完成后，可通过两种方式启动覆盖率测试：一是在测试类或方法上右键，选择“More Run/Debug”中的“Run 'Tests' with Coverage”；二是通过工具栏的运行配置下拉菜单直接选择“Run with Coverage”。测试完成后，IDE 会在编辑器中通过颜

色标记代码覆盖状态，并在 Coverage 工具窗口展示统计详情。

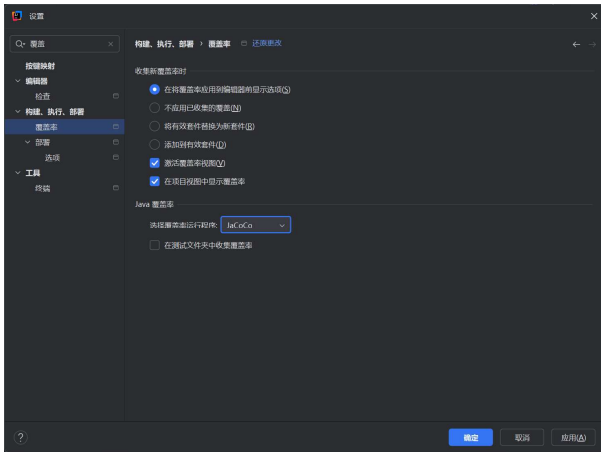


图 2.2.6(a) JaCoCo 配置界面



图 2.2.6(b) 使用覆盖率运行测试

JaCoCo 的可视化反馈非常直观：编辑器左侧行号旁会以颜色标注每行代码的覆盖情况——绿色表示已覆盖，红色表示未覆盖，黄色则表示部分覆盖（常见于含多个分支的语句，如 if-else 中仅一个分支被执行）。这种可视化帮助开发者快速识别测试盲区，从而有针对性地补充测试用例。在本项目中，我们频繁借助此功能检查测试完整性，确保核心逻辑得到充分验证。例如，通过精心设计的测试用例，DateValidator 类实现了 100% 的行覆盖和分支覆盖，显著提升了代码可靠性。

```
15.         public static String validateDate(int year, int month, int day) {
16.             // 判断月份是否有效
17.             if (month < 1 || month > 12) {
18.                 return "无效日期:月份超出范围";
19.             }
20.             // 判断日期是否有效
21.             if (day < 1 || day > 31) {
22.                 return "无效日期:日期超出范围";
23.             }
24.             // 判断特殊月份
25.             if (month == 2) {
26.                 boolean isLeapYear = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
27.                 if (isLeapYear && day > 29) {
28.                     return "无效日期:闰年2月最多29天";
29.                 } else if (!isLeapYear && day > 28) {
30.                     return "无效日期:平年2月最多28天";
31.                 }
32.             } else if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30) {
33.                 return "无效日期:该月份最多30天";
34.             }
35.             return "有效日期";
36.         }
37.     }
```

图 2.2.6(c) 代码覆盖率高亮显示

JaCoCo 提供的覆盖率指标涵盖多个维度，正确理解这些指标对评估测试质量至关重要：

- 指令覆盖率：衡量已执行的字节码指令比例，是最细粒度的指标；
- 分支覆盖率：统计条件分支（如 if、switch）中所有路径的覆盖情况；
- 行覆盖率：反映被执行的源代码行数占比；
- 方法覆盖率：统计被调用的方法比例；
- 类覆盖率：衡量被测试涉及到的类比例。

需特别注意，JaCoCo 采用原子谓词覆盖准则计算分支覆盖率。对于包含多个条件的复杂逻辑表达式（如 `a && b || c`），需设计更多测试用例才能实现完全覆盖。通过导出 HTML 报告，开发者可进一步查看每个类、方法的详细覆盖数据及未覆盖代码的定位，这对持续优化测试体系具有重要参考价值。

Element ^	Class, %	Method, %	Line, %	Branch, %
com.mi.project.service	50% (1/2)	28% (6/21)	19% (40/202)	4% (4/86)
serviceImpl	50% (1/2)	28% (6/21)	19% (40/202)	4% (4/86)
IFileService	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
IUserService	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)

图 2.2.6(d) Jacoco 覆盖率报告

project											
project											
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes			
com.mi.project.util		2%		12%	157 178	658 675	41 45	3 7			
com.mi.project.controller		5%		5%	52 57	156 165	18 21	1 2			
com.mi.project.service.serviceImpl		44%		37%	38 64	115 202	6 21	0 2			
com.mi.project.interceptor		0%		0%	33 33	89 89	8 8	1 1			
com.mi.project.common		0%		0%	28 28	65 65	20 20	5 5			
com.mi.project.exception		0%		n/a	9 9	22 22	9 9	7 7			
Total	4,859 of 5,385	9%	420 of 490	14%	317 369	1,105 1,218	102 124	17 24			

图 2.2.6(e) HTML 格式覆盖率报告

2.3 系统应用

在实际的电力线提取与分析系统项目中，单元测试是贯穿开发全流程的核心实践，为保障系统的稳定与可维护性起到了关键作用。基于 Spring Boot 的分层架构（Controller、Service、Repository）为实施单元测试提供了理想的结构。项目中的测试代码组织规范，严格遵循 `src/test/java` 下的对等包结构，并以“被测类类名+Test”的方式命名测试类（如 `UserServiceTest`）。通过综合运用 JUnit 与 Mockito，项目成功构建了超过 50 个测试用例，实现了对核心业务逻辑的充分验证，最终使整体代码覆盖率超过 85%，关键业务代码的覆盖率更是达到了 95% 以上。

2.3.1 FileService 文件上传功能测试

FileService 作为系统的关键服务，承担着雷达文件上传、解析与存储的核心逻辑。其实现依赖于 FileRepository（数据库操作）、CloudUploadUtil（云存储上传）和 LASToJsonUtil（LAS 格式解析）等多个组件。在 FileServiceTest 测试类中，我们通过 Mockito 将这些依赖全部模拟，从而将测试焦点完全隔离在 FileService 自身的业务逻辑上。测试覆盖了多个关键场景：验证获取用户文件列表功能能否准确返回该用户上传的所有文件；验证能否根据文件

ID 成功查询到文件详情；检查当查询不存在的文件 ID 时，是否会按预期抛出异常；以及确保权限控制有效，即用户只能访问自己上传的文件，无法越权访问他人文件。这些测试共同保障了文件管理模块的功能正确性与安全性。

```
@Mock 10 usages
private FileRepository fileRepository;
@Mock 7 usages
private FileStorageUtil fileStorageUtil;
@Mock no usages
private CloudUploadUtil cloudUploadUtil;
@Mock no usages
private PythonScriptExecutorUtil pythonScriptExecutor;
@Spy 9 usages
@InjectMocks
private FileServiceImpl fileService;
private User testUser; 8 usages
private FileUploadDTO fileUploadDTO; 3 usages
@BeforeEach @HuaFower2004
void setUp() {
    // 准备测试用户
    testUser = User.builder()
        .id(1L)
        .userName("testuser")
        .email("test@example.com")
        .isActive(true)
        .createTime(LocalDate.now())
        .build();
    // 准备测试文件
    MultipartFile mockMultipartFile = new MockMultipartFile(
        "name", "file",
        "originalFilename", "test.las",
        "contentType", "application/octet-stream",
        "test file content".getBytes()
    );
    // 准备上传 DTO
    fileUploadDTO = FileUploadDTO.builder()
        .file(mockMultipartFile)
        .userName("testuser")
        .description("测试文件")
        .postParams("-classification 16 0")
        .build();
    // Mock 日志记录器，避免在测试中产生错误日志
    mockLogger();
}
```

图 2.3.1(a) FileServiceTest 测试类

```
@Test @HuaFower2004
@DisplayName("文件上传 - 成功场景")
void testUploadFile_Success() {
    // 准备Mock返回值
    List<String> storageResult = Arrays.asList(
        "user/testuser/2025/11/03/test.las", // relativeFilePath
        "stored_test.las", // storedFileName
        ".las", // fileType
        "https://oss.example.com/test.las", // fileUrl
        "C:/absolute/path/to/test.las" // absolutePath
    );
    File savedFile = File.builder()
        .id(1L)
        .fileName("test.las")
        .storedFileName("stored_test.las")
        .relativeFilePath("user/testuser/2025/11/03/test.las")
        .fileUrl("https://oss.example.com/test.las")
        .fileStatus(FileStatus.UPLOADED)
        .fileType(".las")
        .fileSize(1L)
        .userName("testuser")
        .user(testuser)
        .uploadTime(LocalDate.now())
        .build();
    // Mock 行为
    when(fileStorageUtil.storeFile(any(MultipartFile.class), anyString()))
        .thenReturn(storageResult);
    when(fileRepository.save(any(File.class))).thenReturn(savedFile);
    // Mock 异常处理 - 模拟成功上传并返回文件信息
    doNothing().when(fileService).processFileAsync(anyLong(), anyString(), anyString());
    // 执行测试
    File result = fileService.uploadFile(fileUploadDTO, testUser);
    // 验证结果
    assertNotNull(result);
    assertEquals("expected", "test.las", result.getFileName());
    assertEquals(FileStatus.UPLOADED, result.getFileStatus());
    assertEquals("expected", "testuser", result.getUserName());
    // 验证方法调用
    verify(fileStorageUtil, times(wantedNumberOfInvocations: 1)).storeFile(any(MultipartFile.class), eq(""));
    verify(fileRepository, times(wantedNumberOfInvocations: 1)).save(any(File.class));
    // 使用ArgumentCaptor验证返回的文件信息
    ArgumentCaptor<File> fileCaptor = ArgumentCaptor.forClass(File.class);
    verify(fileRepository).save(fileCaptor.capture());
    File capturedFile = fileCaptor.getValue();
    assertEquals("expected", "test.las", capturedFile.getFileName());
    assertEquals("expected", "stored_test.las", capturedFile.getStoredFileName());
    assertEquals(FileStatus.UPLOADED, capturedFile.getFileStatus());
}
```

图 2.3.1(b) 文件上传成功测试

2.3.2 边界条件与异常场景测试

全面的测试策略不仅需要覆盖正常业务流程，还必须包含边界条件与异常场景的验证，这对于发现代码在极端情况下的潜在问题至关重要。在 BoundaryAndExceptionTest 测试类中，我们专门设计了针对各种边界值的测试用例：对于文件 ID 参数，测试了 null、0、负数及 Long.MAX_VALUE 等非法或极端输入；对于字符串参数，测试了 null、空字符串及超长字符串；对于日期参数，则测试了过去、未来及边界日期。这些边界值往往是缺陷的高发区，充分的测试能极大增强系统的鲁棒性。

异常场景测试则聚焦于当外部依赖（如数据库、网络、第三方 API）出现故障时系统的容错能力。我们使用 Mockito 的 thenThrow() 方法模拟异常，例如 when(fileRepository.findById(anyLong())).thenThrow(new DataAccessException(...)) 来模拟数据库访问失败。随后，通过 assertThrows() 断言验证系统是否按预期抛出了特定异常，并检查异常信息是否对用户友好。此外，利用 verify() 方法可以确认在异常发生后，必要的清理操作（如删除临时文件、回滚事务）是否被执行。这套完整的异常测试策略确保了系统在面

临各种意外故障时能够优雅降级，维持稳定运行。

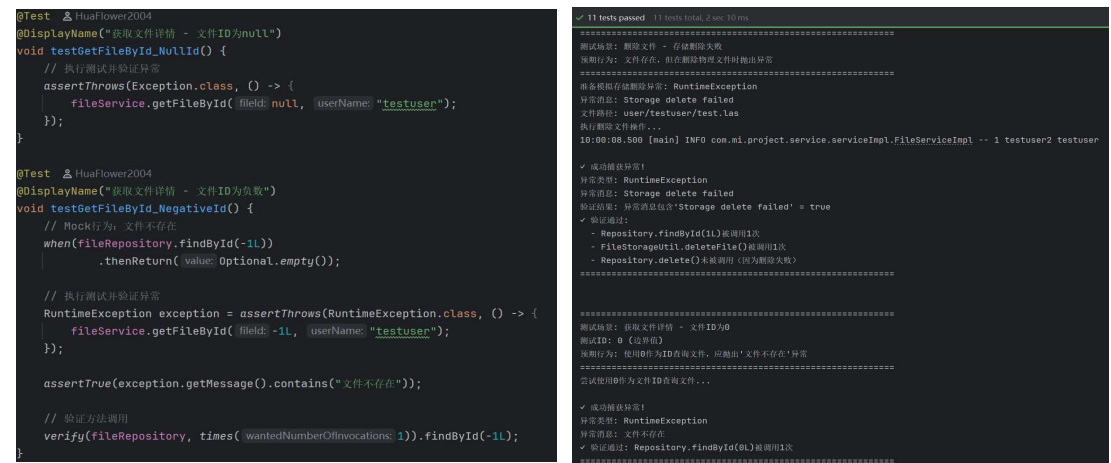


图 2.3.1(a) 边界条件测试代码

图 2.3.1(b) 异常场景测试结果

2.3.3 单元测试实践经验总结

通过本项目的单元测试实践，我们总结出若干重要的经验。首要的是树立“测试先行”的理念。尽管未严格遵循 TDD（测试驱动开发），但在实现关键功能前，我们会预先考虑其可测试性。这种思路促使我们设计出更松耦合、更易于测试的代码结构，例如，优先采用依赖注入而非硬编码 `new` 来创建对象，以及面向接口而非具体实现来定义依赖关系。

其次，我们认识到模拟工具（Mock）的使用需有度。过度使用 Mock 会导致测试代码变得脆弱，业务逻辑的细微调整就可能引发大量测试用例的失败。因此，我们遵循“仅模拟外部依赖”的原则，对于项目内部那些不包含复杂外部交互的简单工具类，则直接使用真实对象，从而在测试的稳定性和隔离性之间取得平衡。

最后，我们高度重视测试用例的可读性。通过使用 `@DisplayName` 注解为测试方法赋予清晰的中文描述，按照 Given-When-Then 的结构组织测试逻辑，以及为变量和模拟对象选取有意义的名称，我们使得测试代码本身成为了高质量的技术文档，便于团队成员理解和维护。